ISLAMIC UNIVERSITY OF TECHNOLOGY

SWE 4301

LECTURE 15

Topic: Abstraction, Encapsulation, Information Hiding and Abstraction Smells

---

February 20, 2025

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1 Abstraction, Encapsulation, and Information Hiding

1.1 Abstraction

Abstraction, as a process, denotes the extracting of the essential details about an item, or a group of items, while ignoring the inessential details. Abstraction, as an entity, denotes a model, a view, or some other focused representation for an actual item. For example, we often hear people say such things as "just give me the highlights" or "just the facts, please." What these people are asking for are abstractions. Of course it is a human smiley face, but

Figure 1: Smiley Face

how do we decide on that answer? We arrive at it through abstraction! There are hundreds of millions of human faces, and each and every face is unique (there are some exceptions). How are we able to cope with this complexity? We eliminated non-essential details such as hair styles and color. We also arrived at the answer by generalizing commonalities, such as that every face has two eyes, and when we smile, our lips curve upwards at the ends.

Abstraction is a powerful principle that provides a means for effective yet simple communication and problem solving. Company logos and traffic signs are examples of abstractions for communication. Mathematical symbols and programming languages are examples of abstraction as a tool for problem solving.

We can have varying degrees of abstraction, although these "degrees" are more commonly referred to as "levels." As we move to higher levels of abstraction, we focus on the larger and more important pieces of information (using our chosen selection criteria). Another common observation is that as we move to higher levels of abstraction, we tend to concern ourselves with progressively smaller volumes of information and fewer overall items. As we move to lower levels of abstraction, we reveal more detail, typically encounter more individual items, and increase the volume of information with which we must deal.

In short, you might say that abstraction dictates that some information is more important than other information, but (correctly) it does not specify a specific mechanism for handling the unimportant information.

1.2 Information Hiding

The technique of encapsulating software design decisions in modules in such a way that the module's interfaces reveal little as possible about the module's inner workings; thus, each module is a 'black box' to the other modules in the system.

We can now identify some of the sources of confusion about the differences between information hiding and abstraction

- **Abstraction can be (and often is) used as a technique for identifying which information should be hidden.** For example, in functional abstraction, we might say that it is important to be able to add items to a list, but the details of how that is accomplished are not of interest and should be hidden. Using data abstraction, we would say that a list is a place where we can store information, but how the list is actually implemented (e.g., as an array or as a series of linked locations) is unimportant and should be hidden. **Confusion can occur when people fail to distinguish between the hiding of information and a technique (e.g., abstraction) that is used to help identify which information is to be hidden.**

- Some of the definitions for abstraction can also be sources of confusion. For example, words like "ignore," "omit," "extract," and "without including" are rather passive, and would not necessarily imply the deliberate hiding of any information, e.g., the information is there, and accessible, but we just ignore it." However, words like "suppress" and "suppressing" present a somewhat different imagequite possibly the active and deliberate hiding of information.

1.3 Encapsulation

"to enclose in or as if in a capsule."

As a process, encapsulation means the act of enclosing one or more items within a (physical or logical) container. Encapsulation, as an entity, refers to a package or an enclosure that holds (contains, encloses) one or more items. It is extremely important to note that nothing is said about "the walls of the enclosure." Specifically, they may be "transparent," "translucent," or even "opaque."

Programming languages have long supported encapsulation. For example, **subprograms (e.g., procedures, functions, and subroutines), arrays, and record structures are common examples of encapsulation mechanisms supported by most programming languages.** Newer programming languages support larger encapsulation mechanisms, e.g., **"classes."**.

If encapsulation was "the same thing as information hiding," then one might make the argument that "everything that was encapsulated was also hidden." This is obviously not true. For example, even though information may be encapsulated within record structures and arrays, this information is usually not hidden (unless hidden via some other mechanism).

It is indeed true that encapsulation mechanisms such as classes allow some information to be hidden. However, these same encapsulation mechanisms also allow some information to be visible. Some even allow varying degrees of visibility, e.g., C++'s public, protected, and private members.

Abstraction, information hiding, and encapsulation are very different, but highly related, concepts. One could argue that abstraction is a technique that helps us identify which specific information should be visible and which information should be hidden. Encapsulation is then the technique for packaging the information in such a way as to hide what should be hidden and make visible what is intended to be visible.

1.4 Reference

https://www.tonymarston.co.uk/php-mysql/abstraction.txt

2 Abstraction Smells

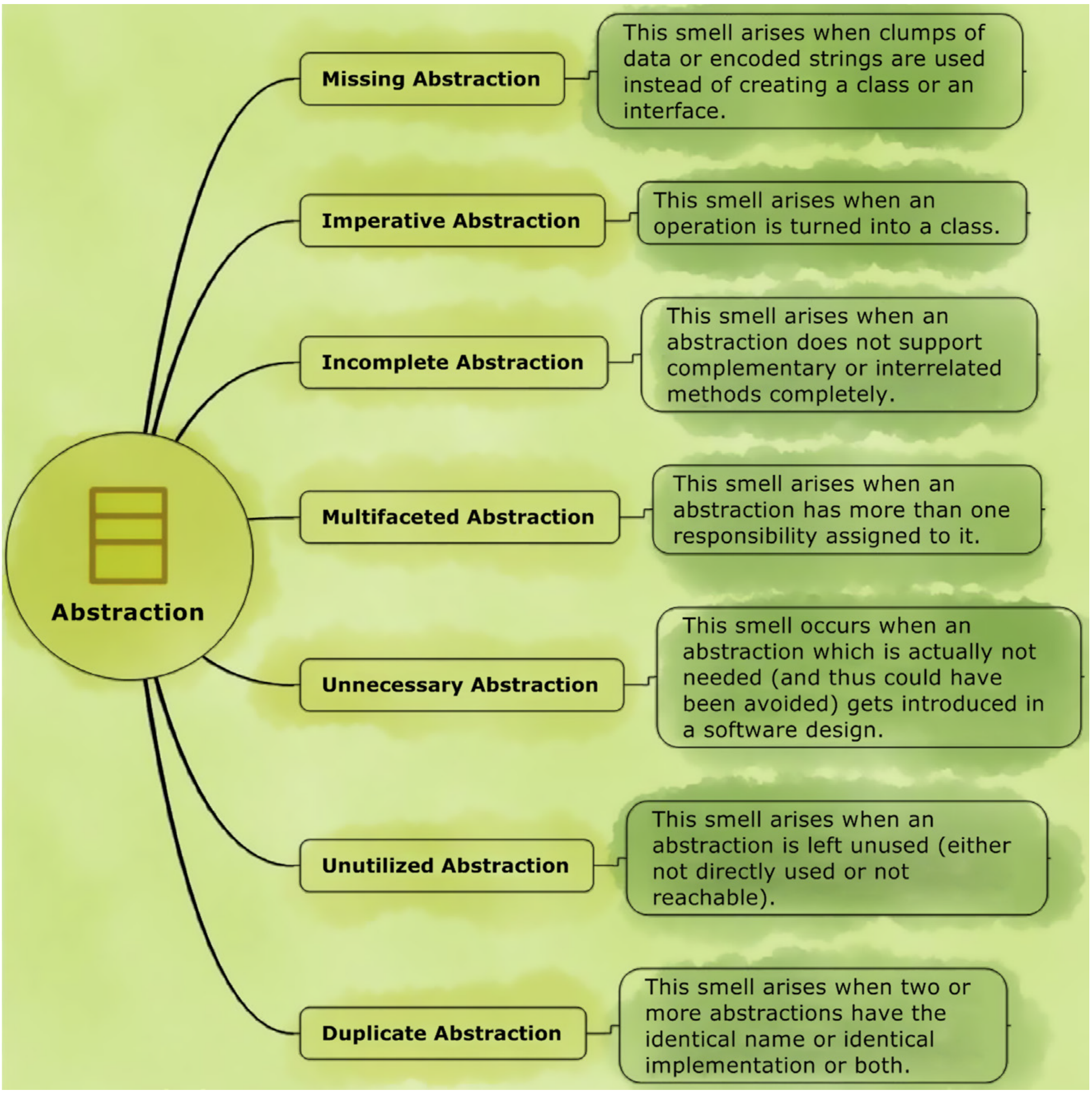

Figure 2: Abstraction Smells

2.1 Missing Abstraction

This smell arises when clusters of data or encoded strings are used instead of creating a class or an interface.

Since the abstraction is not explicitly identified and rather represented as raw data using primitive types or encoded strings, the principle of abstraction is clearly violated. Usually, it is observed that due to the lack of an abstraction, the associated data and behavior is spread across other abstractions. This results in two problems:

- It can expose implementation details to different abstractions, violating the principle of encapsulation.
- When data and associated behavior are spread across abstractions, it can lead to tight coupling between entities.

2.1.1 Exmaple 1:

Consider a library information management application. Storing and processing ISBNs (International Standard Book Numbers) is very important in such an application. It is possible to encode/store an ISBN as a primitive type value (long integer decimal type) or as a string. However, it is a poor choice in this application. Because, ISBN can be represented in two forms10-digit form and 13-digit formand it is possible to convert between these two forms. The digits in an ISBN have meaning; given a 10 or 13 digit number, you can validate whether the given number is a valid ISBN number. It is possible to encode ISBN numbers as strings or as a primitive type value in such an application. However, in such a case, the logic that processes the numbers will be spread as well as duplicated in many places.
Solution:

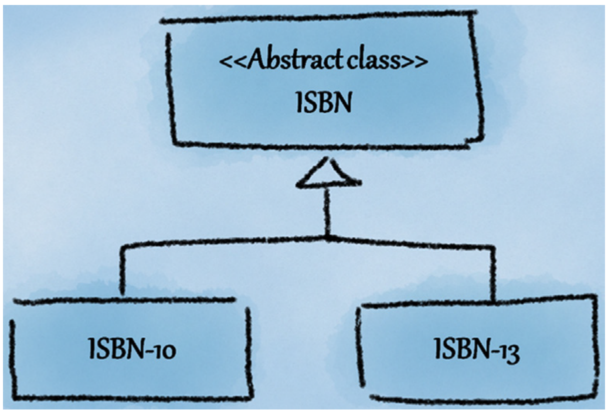

Figure 3: ISBN abstraction structure

2.1.2 Example 2:

Applications are often characterized by clumps of primitive-type data values that are always used together. In many cases, these data clumps indicate a Missing Abstraction. Consider a drawing application that allows a user to select and manipulate a rectangular region of an image. One (naïve) way to represent this rectangular region is to either use two pairs of valuessay variables of double type (x1, y1) and (x2, y2)or variables of double type (x1, y1) and (height, width).

Solution Revisiting the example of the drawing application, a refactoring suggestion would be to abstract the required fields into a new classsay Rectangle class or SelectedRegion classand move methods operating on these fields to the new class.

If repeating data comprises the fields of a class, use Extract Class to move the fields to their own class.

2.2 Imperative Abstraction

This smell arises when an operation is turned into a class. This smell manifests as a class that has only one method defined within the class. At times, the class name itself may be identical to the one method defined within it.

The founding principle of object-orientation is to capture real world objects and represent them as abstractions. Each class representing an abstraction should encapsulate data and the associated methods. Defining functions or procedures explicitly as classes (when the data is located somewhere else) is a glorified form of structured programming rather than object-oriented programming.

If operations are turned into classes, the design will suffer from an explosion of one-method classes and increase the complexity of the design. Furthermore, many of these methods that act on the same data would be separated into different classes and thus reduce the cohesiveness of the design.

2.2.1 Example 1:

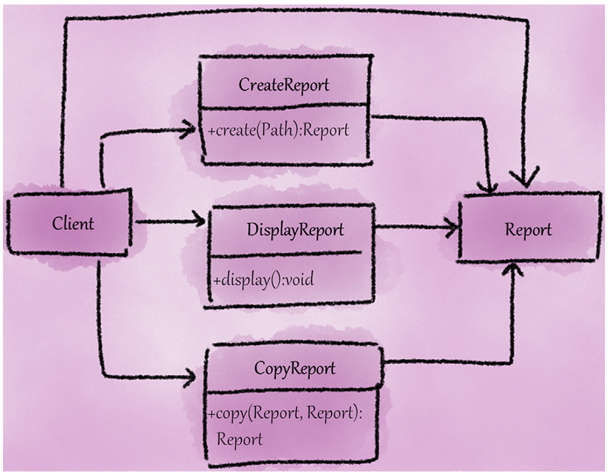

Figure 4: Report Smelly Structure

The smell not only increases the number of classes (in this case there are at least four classes when ideally one could have been used), but also increases the complexity involved in development and maintenance because of the unnecessary separation of cohesive methods.
Solution:

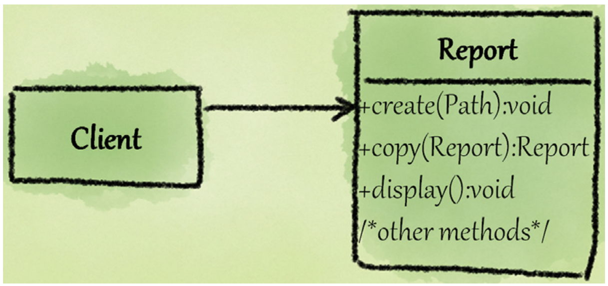

Figure 5: Reort new structure

2.3 Incomplete Abstraction

This bad smell is caused when the abstract does not support all complementary or related methods.

An important abstract implementation is to create a cohesive and complete abstraction. When the abstraction does not support related methods, it may affect the cohesion and integrity of the abstraction. If the abstraction only supports some related methods, its users may have to implement other functions themselves.

Some common complementary methods
Min/Max Open/Close Create/Destroy Get/Set Read/Write Print/Scan First/Last Begin/End Start/Stop Lock/Unlock Show/Hide Up/Down Source/Target Insert/Delete First/Last Push/Pull Enable/Disable Acquire/Release Left/Right On/Off

2.4 Multifaceted abstraction

When abstraction is given more than one responsibility, it will lead to this bad smell.

Why cant there be many aspects of abstraction?
The single responsibility principle states that abstraction must assume a single and clear responsibility and must fully encapsulate the responsibility. When abstraction assumes multiple responsibilities, it means that it will be affected by many reasons and needs to be modified. There is a strong positive correlation between the frequency of design modification and the number of defects.

Refactoring advice
When a class assumes multiple responsibilities, it is not cohesive. You can use the extract class for refactoring.

2.5 Unutilized abstraction

This kind of bad smell is caused when the created abstract is unused (not directly used or inherited).

- Unreferenced abstractionsConcrete classes that are not being used by anyone
- Orphan abstractionsStand-alone interfaces/abstract classes that do not have any derived abstractions

Remove unused abstractions from the design. For APIs that may be used by the client, direct deletion is not feasible, and these abstractions can be marked as expired or discarded, explicitly indicating that they should not be used in newly developed clients.

2.6 Duplicate Abstraction

This bad smell is caused by two abstract names, implementations, or both.

- Same name
  Two different abstract names will affect comprehensibility.

- Realize the same implementation
  Multiple abstract member definitions are semantically identical, but the same elements in these implementations are not captured and used in design. In the inheritance structure, if the implementation of multiple sibling abstractions is the same, it may mean that there is an "unmerged hierarchy" bad smell.

- Name and implementation is the same

Why cant there be repeated abstractions?
Duplicate code is the first of all evils in software. So we have to try to avoid duplication.

If multiple abstract names are the same, it will affect the comprehensibility of the design: the client code developer will not know which abstract to use.

If multiple abstract implementations are the same (code is the same), it will be difficult to maintain: when modifying one of the abstract implementations, it is often necessary to modify the implementation of all other duplicate abstractions. This not only increases the burden of modification but also introduces tiny bugs that are difficult to find. In order to narrow the scope of the revision, it is necessary to avoid duplication as much as possible.

Refactoring advice
For repeated abstractions with the same name, you can change one of the abstractions to a different name.

For implementing the same repetitive abstraction, if the implementation is identical, you can remove one of the abstractions. If the implementation is slightly different, the same implementation can be merged into another class: this can be a base class in a hierarchy, or it can be an existing class or a new class that can be referenced or used by a repeated abstraction.